

A MYTH ABOUT AI SYSTEMS

“Magic pill”: the model answers more accurately if you ask it to act as an expert and complicate the prompt^[1,2].



**Complicate the prompt for accuracy:
“you are an expert lawyer, reason step by step”**



**Pick the architecture for the task:
context, RAG or a tool**

Where an AI system's reliability actually comes from — we work through it across the whole lecture.

LECTURE 3

AI system architectures: agents, RAG, API

03

Which architecture to pick for the task —
and when the right answer is “not AI”



the “magic pill” does not exist

Lecture route — six sections.

One load-bearing line: picking the architecture for the task — and when the right answer is “not AI”.

0



Opening

framing the problem: where reliability comes from

1



Prompt and its limits

what one call can do and where its ceiling is

2



RAG

external knowledge into context — and where it quietly breaks

3



Fine-tuning

changing weights for behavior, not for knowledge

4



Agents

loop, equipment, memory, security — and failures

5



Framework

ladder + checklist: how to choose fast and with grounds

From Lecture 2 we take two concepts as given.

We are not re-explaining — just recalling, so we can lean on them further. Everything else we build up along the way.



Single call (single-shot)

One pass of the model: gave a prompt → got an answer.
No memory between calls, no reaching outward.

The model knows only what is in the prompt and in the weights



Semantic search on embeddings

Text → a vector of meaning; fragments close in meaning are close vectors. Search “by meaning”, not by exact word.

RAG stands on this (Section 2)

Both concepts are the foundation from Lecture 2. Everything we build today (RAG, tools, fine-tuning, agents) leans on them.

THE CENTRAL QUESTION OF THE LECTURE

I have a task and access to an LLM. Which architecture do I pick — and when is the right answer “not AI”?

6

Multi-agent

several coordinated agents

more
complex ↑

5

Agent

loop: plan → act → check → iterate

4

Workflow

predefined paths

3

RAG / context engineering

retrieval-augmented generation

2

One LLM call

prompt; + CoT (step by step), + examples in the prompt (few-shot)

1

Plain code (no AI)

the reference point

simpler ↓



Climb to the next step — only when the task demands something the current one does not cover.

The ladder is a map of the lecture, not a demand to grasp everything now. We work through each step separately.

SECTION 1

The prompt and its boundaries

We saw the whole ladder — now from the bottom: what one call can do and where its ceiling is, before complicating anything.

one precise cut · 3 cases · 1 failure

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

How the task was framed before the prompt — and what that changes.

Before large models, "make the system do what you need" meant not a wish in natural language, but a precise specification and a deterministic program.



Precise specification / requirements

Pre- and postconditions, invariants, acceptance criteria (Z notation, TLA+, Design by Contract). The result is deterministic and verifiable.



Imperative vs declarative

"How to do it" (a step-by-step algorithm) vs "what to get" (SQL/Prolog describe the result, the engine decides how).



Interface contract

A precise input/output agreement (type signatures, OpenAPI, Protobuf). One correct meaning and a way to check it.

WHAT TO KEEP

The discipline of precise framing: a precise prompt is the same requirements spec in natural language. For the deterministic and verifiable (arithmetic, schema validation, rule-based routing) — use classical code, not a prompt.

Bridge: a prompt frames the task for a probabilistic system in natural language; hence both its power (no need to specify the un-specifiable) and its boundary (no single meaning, no determinism).

By default — one call with a good prompt.

The model knows only what is in the prompt (plus what was in the weights at training). It has nowhere else to draw from.



One LLM call with a good prompt

- lowest cost (a single pass)
- lowest latency (no extra round-trips)
- highest predictability (no loops, no retrieval that quietly degrades)

“JUST IN CASE” — ESCALATION



Do not complicate the architecture without a reason expressed in the task^[1] requirements. This is a distribution of the burden of proof, not primitivism.

[1] Anthropic — Building Effective Agents (“find the simplest”)

A role in the prompt tunes tone — not accuracy.

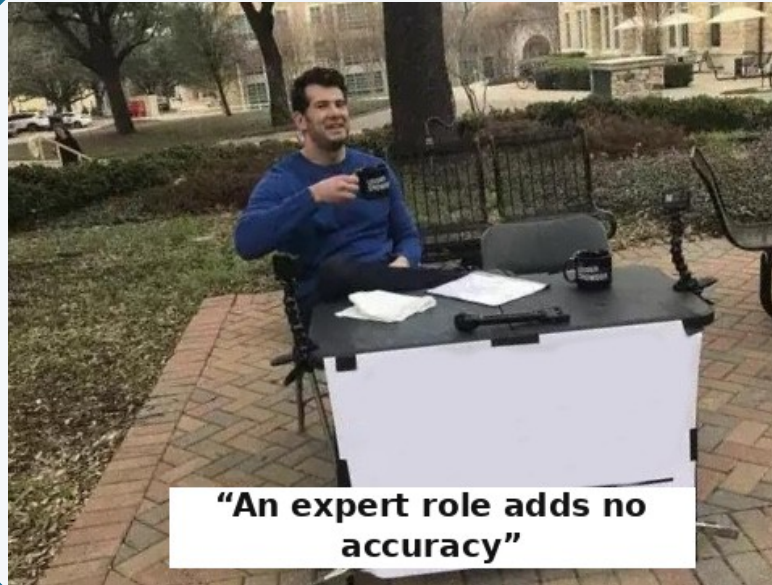
“You are an experienced lawyer” shifts the model's attention toward the text of such a role — but that is about style, not facts.

A common myth

“An expert role in the prompt — and the model is more accurate on facts”

Experiment

**162 personas, 2410 questions —
personas did NOT raise accuracy^[1].**



A role affects:

the tone and depth of the delivery — how formal and detailed, not the truth of a fact.

Need accuracy? The tool is not a role, but context and RAG (a verifiable source). Another “magic pill” point refuted by measurement.

"Role" is two different mechanisms. Don't confuse them.

A role-persona ("you are a lawyer") tunes the tone. The protocol role system/user/assistant is markup of "who is speaking" in the token stream. The second is constantly mistaken for the first.

{ } Chat template: a list of roles → a flat token stream

```
<|im_start|>system behavior rules <|im_end|>  
  
<|im_start|>user question <|im_end|>
```

Special markup tokens (ChatML, Llama 4) are assembled by the template — it can be read and spoofed.

system priority — a tendency, not a boundary

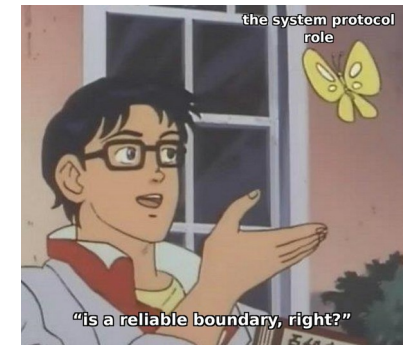
GPT-4o obeys the priority ~**63.8%** — still ≠ 100%: it's a tendency.



STI / role spoofing: the role is faked

The string `<|im_start|>assistant` from external text → ChatInject ASR
5.18% → 32.05% (Llama-4 up to 88.3%).

What to do: don't design defenses assuming "the system always outranks the user". Escape special tokens in INCOMING external content and check the chat template of local models — a foreign template quietly breaks the priority.



The full walkthrough of prompt injection as a class of agent attacks is in the agents section (security).

Prompt structure: separate instruction, context, data.

A flat prompt forces the model to guess where the instruction ends and the data begins. An explicit boundary removes that ambiguity.



Instruction

what to do



Context

on what basis



Data

what exactly to work with

Delimiters

- XML tags: `<instruction>...</instruction>`
- Markdown headings: `## Task · ## Data`
- Triple quotes / backticks around the data

The same principle as structured output (Section 4): there a schema sets the structure of the OUTPUT, here delimiters set the structure of the INPUT. A clear structure instead of “infer the form by meaning”.

A model that has been shown where the instruction ends and the data begins less often takes a fragment of data for a new command — the same confusion underlies prompt injection (Section 4).

Chain-of-thought helps — but you cannot audit it.

Without CoT

“There were 23 apples, 7 spoiled, then 2 crates of 6 were bought.
How many good ones?”

→ a plausible but wrong number

With CoT (“reason step by step”)

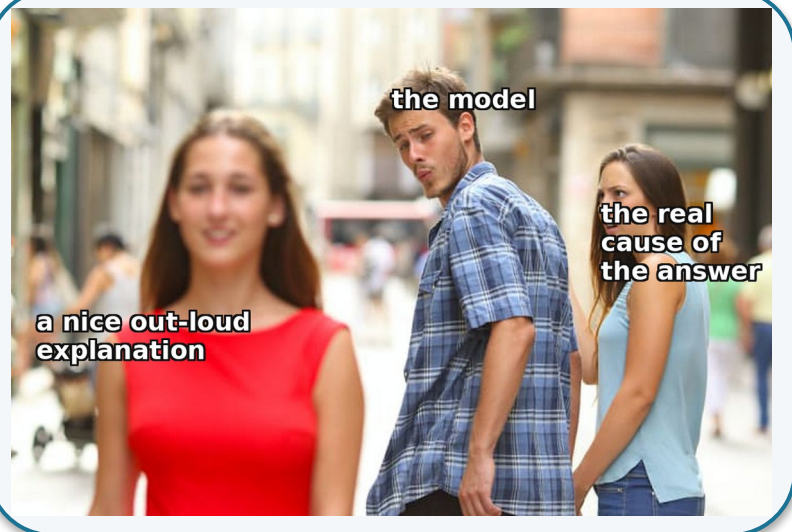
$23 - 7 = 16^{[1]}$ · $2 \times 6 = 12$

$16 + 12 = 28$

→ correct

Technically this is still one call — CoT is a tool for a class of tasks (a chain of steps), not a global switch.

But the stated reasoning need not reflect the real cause of the answer^[2] (low explanation faithfulness):



Check the result, not the model’s self-explanation.

Verify facts against an external source — a plausible reasoning text confirms nothing.

Reasoning out loud is not an audit log.

Faithfulness (of the explanation) — how well the spoken chain reflects the real cause of the answer. It was measured directly — and it is low.

Anthropic experiment (Apr. 2025)



The task is given twice: without a hint and with a hint in the prompt that changes the answer ("the professor thinks it's C")



They look: when the answer changed under the hint — did the model mention it in its reasoning?



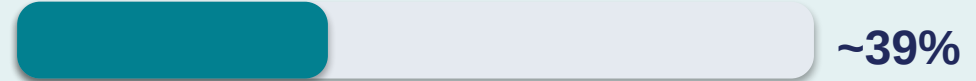
Often the model changes the answer, but builds a DIFFERENT, invented rationale, without naming the real cause

How often the model admits the hint

Claude 3.7 Sonnet



DeepSeek R1



And worse — on hard tasks (GPQA below MMLU): where audit is needed most, the explanation can be trusted least.

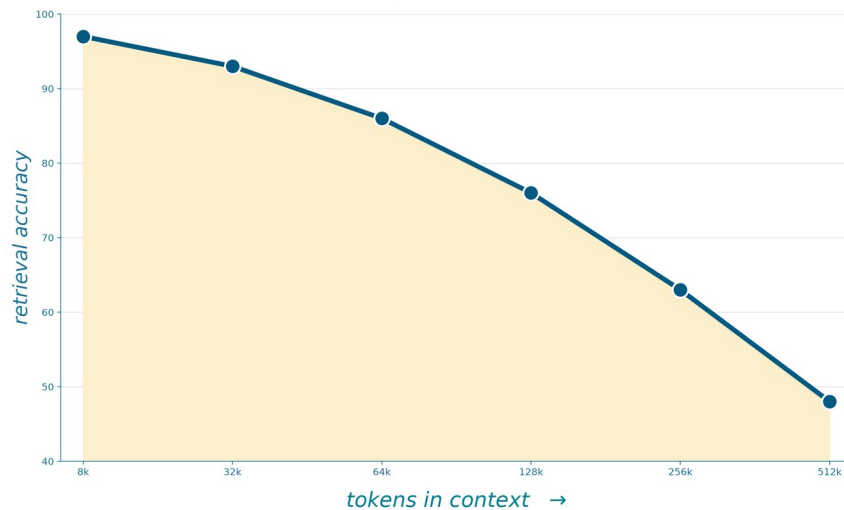
What to do: a human validator checks the RESULT against an independent source (a database, document, calculation, expert), not the attached "chain of reasoning". Control based on the model's self-explanation is not control.

Context engineering: the minimum of high-signal.

Prompt engineering is one instruction. Context engineering is curating the whole set of tokens the model sees at inference.

Retrieval accuracy ↓ as the context grows

SCHEMATIC — illustration of the effect, not measured data · effect: Chroma Research, 2025



context rot = the same “lost in the middle” from L2 — a new term, not a new entity^[1,2]

When NOT RAG (point 1)

a small, stable corpus that fits in the window →
full context + prefix caching, not RAG
infrastructure



RAG here would add fragility without a payoff

“Find the smallest set of high-signal tokens that maximizes the probability of the desired outcome” — this is an engineering requirement, not aesthetics^[3].

Cheat sheet: how to build a prompt.

The minimum below which a prompt systematically underperforms. Apply it to any prompt before the first run.

1 Role

if you need tone/register — and NOT as a promise of accuracy

2 Task

a concrete verifiable action, not a vague wish

3 Context

the minimum needed, not “everything that might come in handy”

4 Output format

stated explicitly if the answer is machine-processed

5 Delimiters

if there is more than one kind of content (instruction/data)

6 Examples (few-shot)

only if the format is not obvious from one instruction

7 CoT

only if the task needs multi-step reasoning

8 Length

no longer than needed — extra tokens “drown” in the context

A compact form of the whole section: role $\neq$ accuracy, structure helps separate inputs, CoT is a targeted tool, context is minimal. For large architectural decisions (RAG / FT / agent) — the eight-step checklist of Section 5.

SECTION 2

RAG: retrieval-augmented generation

Retrieve the relevant → put it in context → answer grounded in a source

external knowledge · 3 cases · 1 failure

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

2

How we searched text before embeddings.

The R in RAG is retrieval, search: a discipline with half a century of history. Exactly how it works determines where RAG works and where it breaks.



Inverted index

For each word — a list of documents where it occurs (a machine catalog). The foundation of Lucene, Elasticsearch, PostgreSQL full-text.



Boolean search

A query as a logical expression ("error AND authentication NOT tomcat"): precise, predictable, explainable selection.



TF-IDF → BM25

Ranking by word importance (rarer in the collection — stronger signal). BM25 (Okapi) — a cheap, explainable baseline many do not beat.

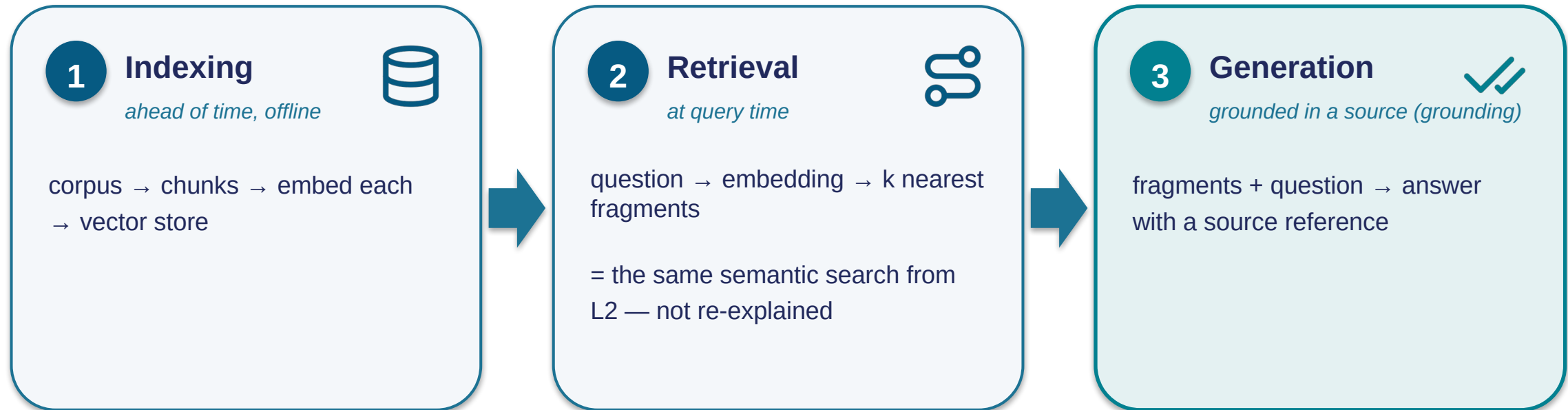
WHAT TO KEEP

The classics are precise on codes and identifiers, where semantic search blurs. A strong RAG-2026 is a hybrid of BM25 + dense vectors, lexical filters over metadata, ranking discipline (a reranker) and observability: recall/precision on a golden set.

Bridge: semantic search over embeddings (Lecture 2) adds semantic matching on top of the classics instead of lexical — but does not replace it. RAG extends classical search, it does not abolish it.

The RAG principle — three steps.

RAG = indexing → retrieval (semantic search from L2) → grounded generation; “I don't know” is a correct answer.



“I don't know” / “see source X” is a correct answer from a RAG system^[1]. A plausible answer on an irrelevant retrieval is a defect, not “at least something”.

RAG-2026: agentic RAG by default · hybrid search (BM25 + dense vectors) · reranker · miss cascade **5.7% → 1.9%** (Contextual Retrieval).

[1] [Lewis et al. 2020 — Retrieval-Augmented Generation \(NeurIPS\)](#)

When RAG is the right choice.

RAG is justified when the signal on the features below is strong^[1] — and there are no blockers from the next slide, “when NOT RAG”.

Large / growing

does not fit the window whole, or
it is costly to put the entire corpus
in every request

Changing

documents, prices, regulations
update more often than model
versions ship

Freshness + provenance

the answer relies on a verifiable
source; you can show where the
fact came from

Private base

the company's knowledge is not
in the weights of a public model

The features reinforce each other → RAG

A corporate base of thousands of regulations, updated weekly, an answer with a mandatory reference to the clause: all features converge, no simpler mechanism closes them jointly → a textbook RAG profile.



One feature is a reason to take a look^[2], not to build automatically: check the blockers on the next slide.

When RAG is NOT the right choice.

Knowing when RAG is not needed is worth more: it is a fashionable architecture, put where it does harm.

1 The corpus fits in the window



a rough guide — under ~200k tokens, rarely changes

→ **full context + prefix caching, not RAG infrastructure**

2 A fixed policy / value^[2]



a fare, a price, a clause of a regulation, a rule

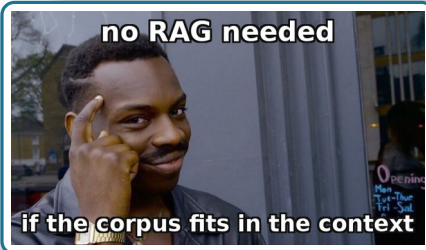
→ **a deterministic lookup table / a static page**

3 Data available in real time via API / MCP



in an internal service, a database, another system's search

→ **call the tool directly; a RAG index on top is an extra layer, more fragile and more stale**



A common anti-pattern — putting RAG infrastructure where the whole corpus comfortably fits in the context window and rarely changes.

RAG is redundant if ANY of the three holds^[1]: (a) the corpus fits in the window and is stable, (b) the task reduces to a fixed value, (c) the knowledge is already available directly and live via a tool. “A direct call returns data as of the request; a RAG index — as of the last indexing.”

^[1] Red Hat — [RAG vs Fine-Tuning \(when not RAG\)](#) · ^[2] McCarthy Tétrault — [Air Canada \("generation on top of a fixed policy"\)](#)

The failure of RAG at scale.

“Returned something” ≠ “returned the right thing”. RAG has no ^[1,2] “not found” signal — it always returns the k nearest, even irrelevant ones.



Legal-AI

the “nearest” cases from another jurisdiction / an overturned precedent — the model relies on them as fact



Medical-RAG

mixed fragments of different patients — close in symptoms, but clinically cannot be combined



Support bot

worked on hundreds of articles; after growing to thousands, quality quietly degraded — nobody noticed

Air Canada — architecture walkthrough ^[3]

Diagnosis

generated plausible text was placed in a role that required a retrieved, verified fact — a failure of grounding

The right alternative

a fixed policy → a lookup table / page; need a dialogue → RAG with strict grounding, a mandatory citation, an explicit “I don't know”, a human check

SECTION 3

Fine-tuning vs prompt vs RAG

We solved the knowledge problem through RAG. But what if the problem is not knowledge, but the model's behavior — its tone, format, policy?

behavior tuning · 4 cases · 1 failure

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

How the ML task was solved before large models.

Fine-tuning a large model is not an exotic novelty of a new era, but a direct continuation of the classical transfer-learning scheme. Let's rebuild it from scratch.



Sample + ground truth

A labeled set "input → correct answer" (ground truth, reference labeling); the model learns to reproduce it and generalize to new data.



Train / val / test split

Three disjoint parts: you train on train, tune on validation, measure once on test. The rule: you must not test on the training data.



Transfer learning

Take a model pretrained on a large corpus and cheaply fine-tune it for your narrow task: faster and more accurate than from scratch ("pretraining → fine-tuning").

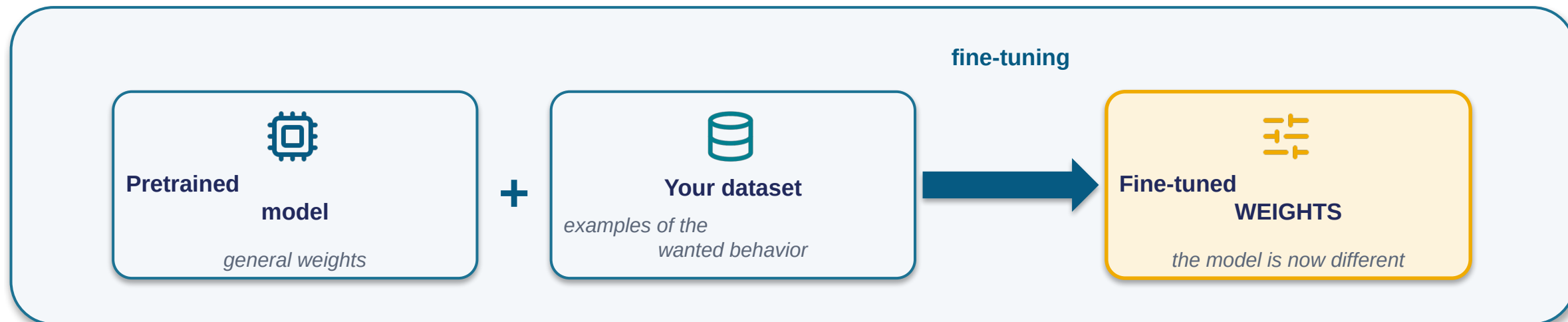
WHAT TO KEEP

Eval sets (golden set) — without them catastrophic forgetting is invisible; **versioning of data and weights for rollback**; **train/test discipline against leakage**; **drift monitoring**. LoRA made the fine-tuning step cheaper, but not the discipline around it.

Bridge: PEFT/LoRA is the same transfer learning, pushed to the limit of cheapness and on top of an incomparably larger model. The idea is not new — what became new is the scale of the base model and the cost of the step.

What fine-tuning is.

Fine-tuning is continuing to train an already-built model on your data. In L1 — a type of use; here — an architectural choice, one of the rungs of the ladder.



Prompt / RAG → CONTEXT

They change only the input — the weights are untouched; the effect lives only within the request.

Fine-tuning → THE WEIGHTS

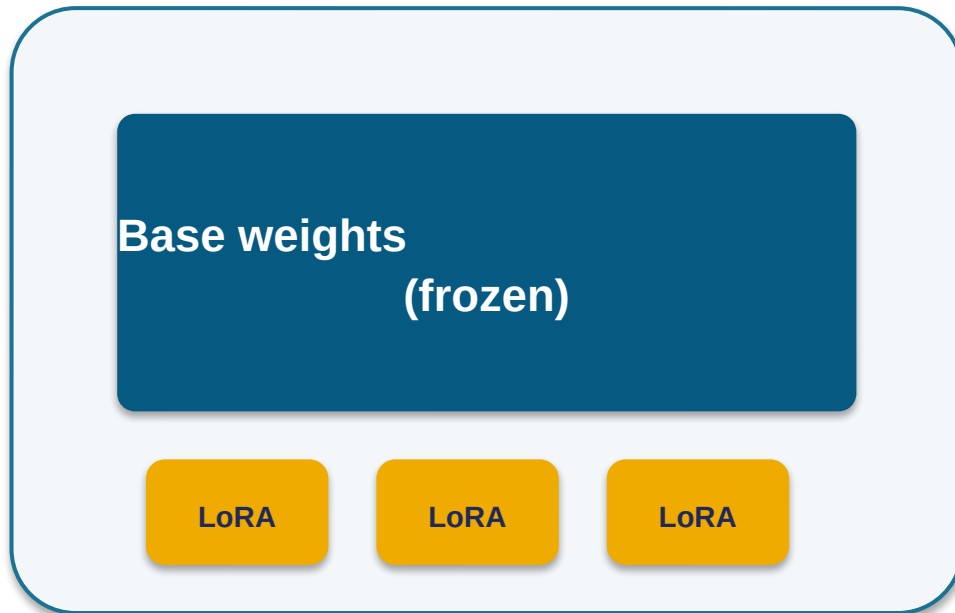
The change is baked into the model — it applies always and costs more than what context changes.

Prompt/RAG = “what to show the model”. Fine-tuning = “change the model itself^[1]”. In practice “fine-tune” almost always means LoRA/PEFT, not retraining all the weights (next slide).

[1] IBM — RAG vs Fine-Tuning (fine-tuning changes the weights)

PEFT instead of full fine-tuning.

PEFT — the base weights are frozen, only a small set of adapters is trained. LoRA — low-rank adapter matrices; QLoRA — the same on top of a quantized ^[1,2] model.



adapters — megabytes vs gigabytes

1. Cheaper and faster

millions of parameters instead of billions;
QLoRA — on a single GPU

2. Modularity

adapters are megabytes vs gigabytes;
one base — many specializations

3. ↓ Forgetting risk

the base is frozen, physically not
overwritten by the new signal



PEFT / LoRA: cheap,
modular, lower forgetting
risk



Full fine-tuning of all
weights in 2026 —
almost never

98.4% of PEFT-tagged models are LoRA^[3]

out of 20,834 cards on the Hugging Face Hub · caveat: the share among PEFT-tagged models, not among all fine-tuning

PEFT (LoRA/QLoRA) is almost always preferable to full fine-tuning: cheaper, more modular, ↓ forgetting risk. Full fine-tuning in 2026 — almost never.

What here is knowledge, what is behavior, what is deterministic.

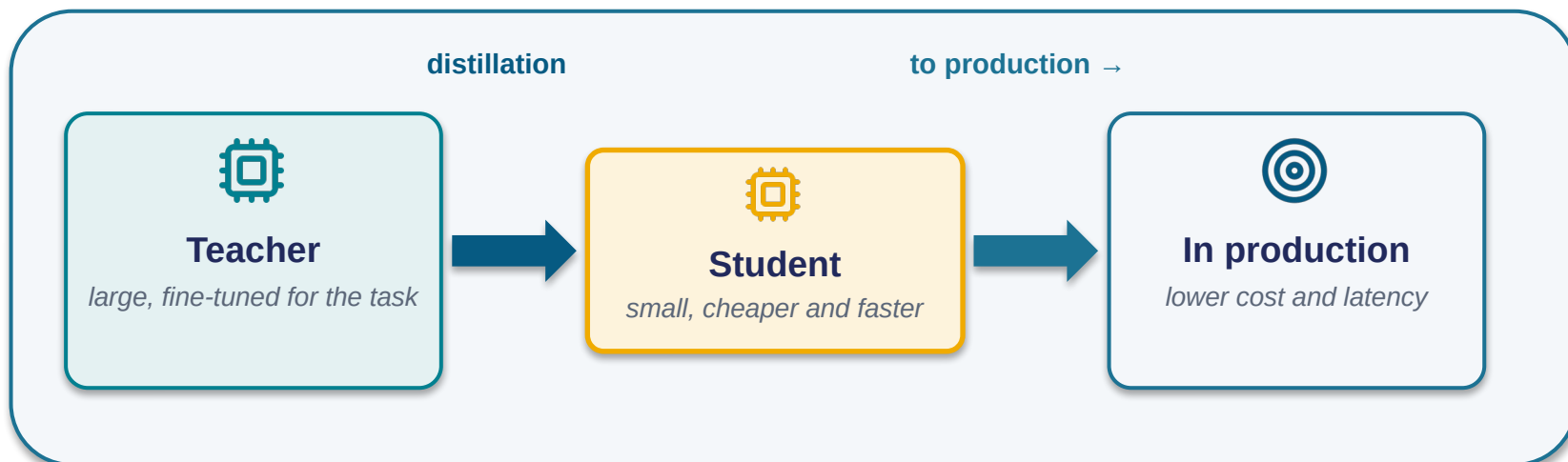
The design question is not "RAG or fine-tuning". Split the problem along axes: knowledge → RAG, behavior → PEFT, cheaper → distillation, deterministic → code.

If the task requires...	→ the right tool	...and NOT this one, because
knowledge changes / freshness, provenance needed	RAG (or a long context for a small corpus)	not fine-tuning: knowledge will go stale, retraining is expensive, risk of forgetting
stable behavior / tone / format / policy	fine-tuning (PEFT)	not RAG: it feeds knowledge into the context but doesn't change the model's manner
reduce cost / latency on a narrow task	fine-tune a teacher + distill into a student	two separate techniques in tandem; prompt/RAG don't shrink the model size
a deterministic, verifiable answer	plain code, no AI	neither RAG nor FT: AI adds non-determinism with no gain

What to do: a hybrid is the norm WHERE the task has BOTH a knowledge problem AND a behavior problem. No behavior problem — no FT needed, even when there's RAG. Each component is added to meet its own requirement — the same ladder rule.

Distillation is not a kind of fine-tuning — it is a separate technique.

Fine-tuning changes the model's behavior^[1]. Distillation is compression: transferring the skills of a large model into a small one. They are often confused, but these are two taxonomically distinct operations that work together.

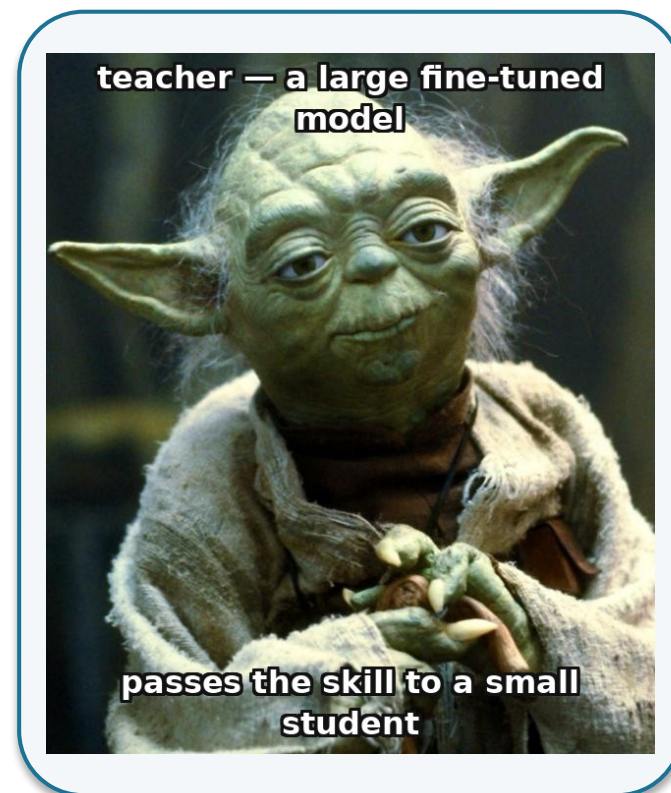


Fine-tuning answers:

“how the model behaves” — tone, format, policy.
Changes the WEIGHTS for behavior.

Distillation answers:

“how to make the model cheaper” — the same skill
in a smaller model. This is compression, not a
change of behavior.



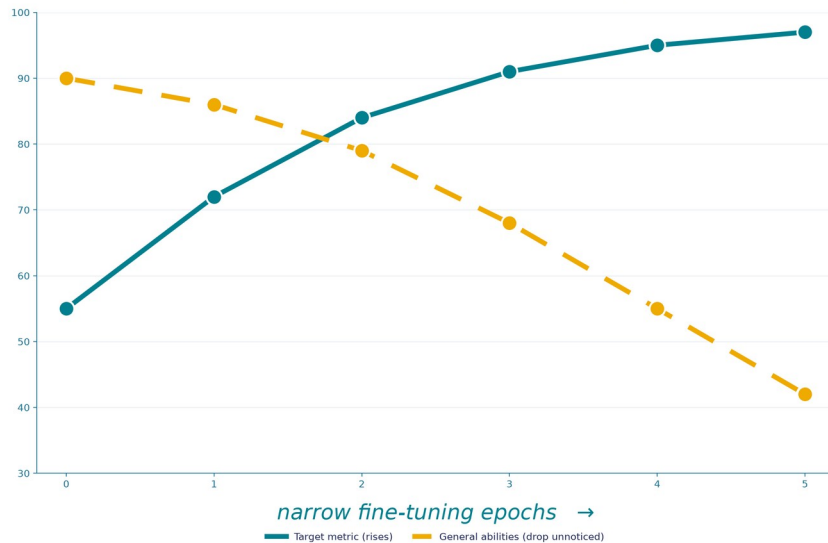
What to do: do not write “distillation is fine-tuning”. In tandem they go like this: first fine-tune the teacher for the task, then distill into the student for cost. The criteria “what goes where” — on the next slide.

Failure: catastrophic forgetting.

Catastrophic forgetting — degradation of the model's general abilities as a result of narrow aggressive fine-tuning.

Narrow aggressive fine-tuning: target ↑, general abilities ↓

SCHEMATIC — illustration of the effect, not measured data · effect: Luo et al., arXiv:2308.08747, 2023



worse as the model scales up — a larger model starts higher, so it has "further to fall"

No eval loop on general tasks + no dataset/weights versioning → you won't see the break before production + you can't roll back → this is not a "risk", it is a criterion for "do NOT fine-tune"

Right way: PEFT (frozen weights — lower risk); for changing knowledge — RAG, not fine-tuning at all.

SECTION 4

Agents

From a conversationalist in a chat window — to a component of a production system: the loop, the equipment, memory, tool access — and where all of it breaks.

agent + equipment · 5 cases · 4 failures

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework



Controlled automation existed long before agents.

"Agent" sounds like an invention of the large-model era, but controlled process automation is a mature discipline. Without it you can't judge what an agent adds and what it breaks.



Finite-state machine

A set of states and event-driven transition rules ("created → in progress → closed"). Possible transitions are visible, impossible ones are excluded by construction.



Workflow engines

Execute a predefined process by a fixed schema: BPMN, DAG orchestrators (Airflow), RPA. The order of steps is defined in advance, not on the fly.



Control loop

plan → act → check as a feedback loop (control theory, SCADA); the military analog is the OODA loop. Not a single action, but a loop with correction.

WHAT TO KEEP

Deterministic workflows where possible; idempotency (repeating a step does not break state); the least-privilege principle for tools; limits and audit of the loop. Reliability practices of the control loop don't go anywhere.

Bridge: an LLM agent is the same plan → act → check control loop, but the steps in it are generated by the model non-deterministically, rather than choosing a fixed automaton rule. Hence both the flexibility and the failures of the section.

The model becomes a system component: API + MCP.



Structured output

output strictly by schema (JSON), not text to parse

embeddable



Tool calling

the model formulates "call X"; your code executes it, not the model

active



Prompt caching

don't recompute the unchanged prefix on every request

economical



MCP — "USB-C for LLM tools"

N×M incompatible integrations → **N+M**^[1]

a tool once = an MCP server; a model once = an MCP client

AI Anthropic
11/2024



 OpenAI
03/2025



 Google
04/2025

Standardizing the connection ≠ security of what you connect — and it makes the trust problem worse.

- code in your environment / access to data
- the description enters the context — a carrier for prompt injection

No mechanism makes the model more reliable — the ladder rule still holds. Ease of connection is not an argument for connecting.

[1] [Anthropic — MCP donation / Agentic AI Foundation \(N×M → N+M\)](#) · Current savings figures and the scale of the MCP ecosystem — see sources.

An agent is not a "pricier chat". It's a different cost class.

The comparison baseline is one chat turn (tens of cents). Each step up multiplies token spend, it doesn't just add to it.

Single LLM call (chat)

the base cost of a request

×1

Single agent (loop)

plan → act → check → repeat: many passes per task

≈50×

Multi-agent

*on top of the agent — coordinating several agents
(Anthropic, 2025)*

another ×15

What to do: budget BEFORE choosing the architecture. Prompt caching (don't recompute the unchanged prefix) in an agent is no longer a "nice reference" but a necessity: without it the ×N loop cost becomes unaffordable.



The multipliers are order-of-magnitude (Anthropic, 2025), not an exact tariff; the absolute price depends on the model and task.

MCP: connecting a tool is now a matter of minutes.



MCP — a connection standard (already covered above)

Describe a tool once as an MCP server — and every model-client sees it. Connecting became a matter of minutes. But the ease of connecting says nothing about what exactly you're connecting.

Read the catalog critically

"up to 90,000 servers" → actually runnable $\approx$ **10,000** = **11%** of the catalog.

The rest are duplicates, broken and stubs. And even the working 10 thousand aren't checked for security.



The trust turn: standardizing the connection $\neq$ security of what's connected — and it sharpens it.

30+ CVE

in a 60-day window against MCP servers (~43% — command injection)

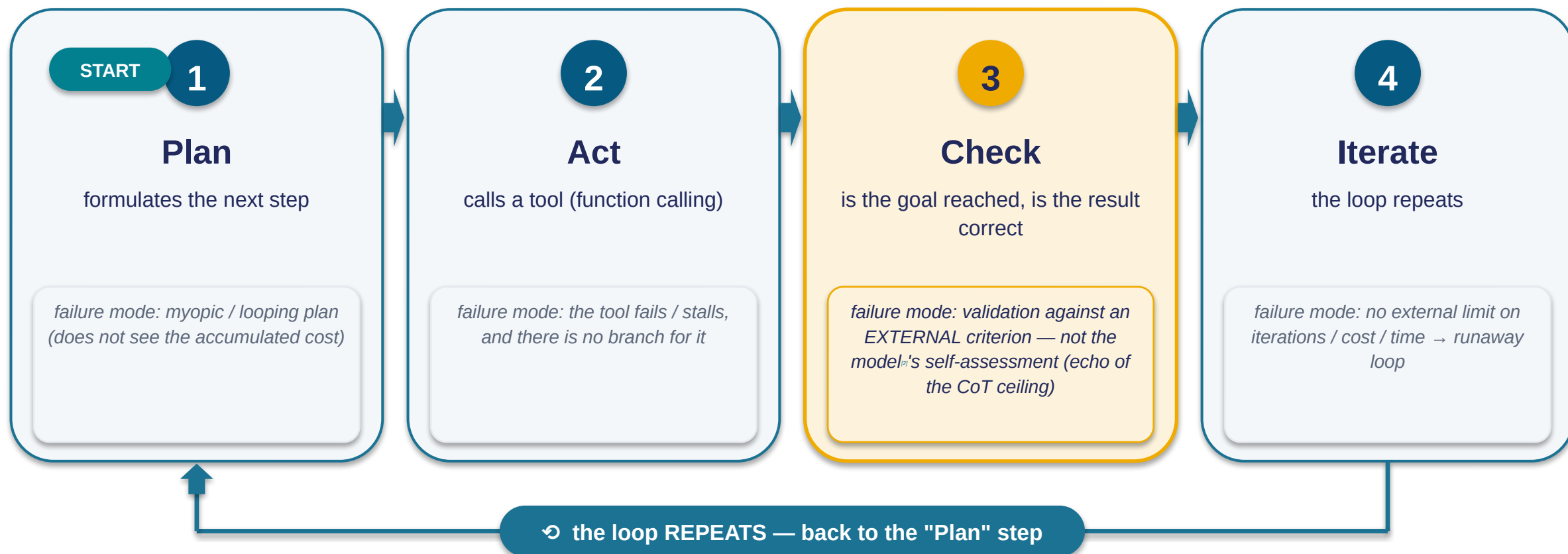
82%

path traversal among 2,614 checked implementations

What to do: ease of connection is not an argument for connecting. Each MCP server = someone else's code in your environment + a carrier of injection into the context. You connect it to meet the task's requirement, with an assessment of the new trust boundary.

The agent loop: plan → act → check → iterate.

An agent is an architecture where the model does not make one pass, but works in a loop, determining the sequence of steps itself^[1].



Workflow vs agent.

Predictable task → workflow^[1]; unpredictable AND the value justifies a multiple-fold increase^[2] → agent.

Workflow

LLM and tools along paths predefined in code

- the sequence of steps is known in advance
- predictable, auditable
- most reliable production systems are a workflow

Agent

the LLM dynamically determines its own process

- the sequence is not fixed in advance
- many times more tokens than a chat
- lower auditability, higher risk of loops

Can I write out the sequence of steps in advance, before launch? yes (even with branches) → workflow · fundamentally no AND the value justifies the multiple-fold cost/risk → agent

Nesting is the norm: a code-review agent calls a "lint → test → format" workflow as one step; a ticket-processing workflow delegates parsing a free-form complaint to a mini-agent. Dynamics — only where it is needed.

Find the simplest option. "Too lazy to formalize" does not make a task unpredictable.

Multi-agent by default is not a win — it is a risk multiplier.

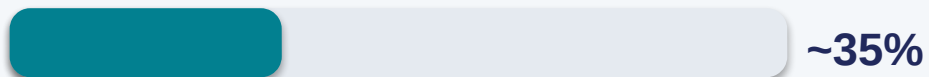
In a chain of n steps, where a failure of any one ruins the result, reliabilities multiply: success $\approx p^n$. The engineering intuition "each step almost always works" is mathematically false.

Chain reliability = p^n

95% × 10 steps



90% × 10 steps



95% per step sounds reliable — but $0.95^{10} \approx 0.60$. More agents = more steps = lower overall reliability.

Topology decides: coordinator > swarm

A "swarm" of equal agents amplifies errors **17.2×**; a single coordinator — only **4.4×** (Zartis/Redis).



Anthropic verbatim: "multi-agent works mainly because it helps spend enough tokens to solve the problem" — the win comes from token volume, not from "coordination magic".

What to do: start with one strong agent. Multi-agent — only if the task splits into WIDELY parallel independent subtasks of high value; otherwise +15× tokens and coordination overhead will not pay off.

Every agent equipment slot is a trade-off, not a win.

A real assistant agent (Claude Code, Cursor, Aider) is a plan → act → check → iterate loop PLUS equipment. Five typical slots^[1]:



Just as you don't climb the architecture ladder without a task requirement — don't add memory, subagents or MCP to an agent "just in case". Each slot carries its own cost: operational complexity, a new failure surface, a new trust boundary — and it must answer a concrete trigger (Section 5).

^[1] agent-harness-registry — map of the agent's equipment slots

Agent memory — the same scale question as RAG.

Memory — what the agent remembers *BETWEEN* sessions (unlike the context of a single conversation). The spectrum runs from a flat file to graph knowledge^[1] bases.



Flat file

the agent appends facts to a text log, reads it whole on startup. Works while the log is small and stable — a direct parallel to the RAG criterion "the corpus fits in the window".



Vector / graph knowledge base

- mem0 — user memory between sessions
- Cognee — memory on a knowledge graph
- Graphiti / Zep — a temporal graph: when a fact is true, when it's stale

The same knowledge-scale question that RAG solves for a document corpus arises here for the agent's own memory. Don't give a graph knowledge base to an agent with short, unrelated sessions — that's the same technical debt without a requirement as RAG for ten articles.



"just append facts to one text log"

a graph knowledge base — only when scale demands it

[1] agent-harness-registry (live-eval) — the agent memory spectrum · Source: the public agent-harness-registry (workain lab, independent evaluation).

"An agent that remembers" — not always for the better.

Having memory intuitively seems like a pure win. Independent evaluation shows: sometimes — dramatically not^[1].



Letta — Tier D

loses to BOTH the bare model AND the flat file on every task.
persistbench_v1:

 Bare model	1.000	94 s
 Flat file	0.833	159 s
 Letta	0.750	496 s

Mechanisms: capitulation under pressure · verbosity drowns the fact · the fact is noticed but not committed.

Caveat: the test is on Letta v0.6.7 — behind the current v0.16.8 (~18 mo).

Anthropic Memory Tool — Tier B

A strong result overall — but even it loses data on

17% of tasks

- an explicit refusal to record an "ephemeral" fact
- an unmotivated "out of scope" refusal
- silent lossy compression — the detail can't be recovered
- non-reproducibility: the same conversation twice → a different result

^[1] agent-harness-registry (live-eval) — Letta Tier D / Memory Tool 17% tail · agent-harness-registry (workain lab, independent evaluation 2026-07-05). "Works well" ≠ "works always" — the same lesson as RAG at scale and forgetting.

An instruction file for an agent is not a "magic vaccine".

The operational layer — convention files (the CLAUDE.md / AGENTS.md class) + a task journal. The intuition "wrote an instruction → and it got more correct" diverges from measurement.



presence paradox

RCT (Gloaguen et al. 2026): the mere presence of an instruction file gives NO significant gain^[1] in success — while cost and step count grow.

Helps only where it actually fills a documentation gap.



Honest Lying

Dixit, Kamal, Oates 2026: self-authored memory can ENTRENCH a wrong belief — the journal cements an early mistake instead of revising it.

The reflection is wrong → retries lean on it.

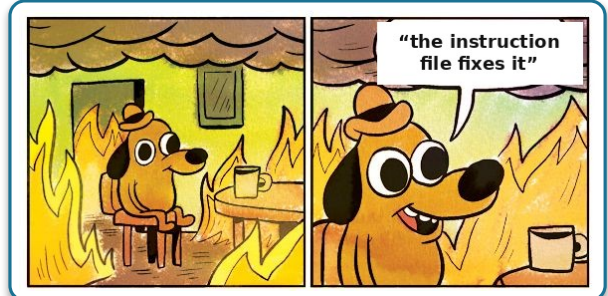


claude-code#51735

A real case: a past mistake acknowledged in writing did NOT prevent its recurrence 25 days later^[2].

A record of a failure ≠ a guarantee the behavior changes.

Useful when it fills a real gap; useless when it duplicates what the model already has; can harm when self-authored memory entrenches a mistake. The same pattern as a role in a prompt: "add X → it gets more correct" is not confirmed by measurement.



[1] [Gloaguen et al. 2026 — Evaluating AGENTS.md \(presence paradox\)](#) · [2] [GitHub anthropics/claude-code#51735 \(the error repeated after 25 days\)](#)

Skills, subagents, access — and the trust boundary.



Skill

a reusable “how-to” procedure for a recurring task



Subagent

a separate context window: keep the main one clean + isolate the untrusted



MCP access

every connection is a new trust boundary and retention policy



Security: the moment an agent delegates and connects, a surface appears that a single call never had^[2,3]

GitHub MCP: leak via injection (May 2025)

an issue with an embedded instruction + an over-broad token (a PAT for all repos) → the assistant dumped private repositories into a public PR^[1].

Disaster = injection × broad rights. Remove either — the attack fails.

“We have ZDR” ≠ “stored nowhere”: a court order (NYT v. OpenAI) + third-party services/MCP outside ZDR^[4,5]. Regulated data — don't send without ZDR/BAA.

4 design rules

1

Least-privilege

the minimum of tokens/rights

2

Isolate the untrusted

apart from privileges (a subagent)

3

Human in the loop on writes

the irreversible — only via a human

4

Allowlist / version pinning

audited versions; deny by default

"We have ZDR" ≠ "data is protected across the whole chain".

ZDR (zero data retention) covers the core model calls — but not the whole architecture. An agent = a model + tools, and tools are often outside ZDR.



ZDR does NOT cover

- Files API
- code execution in containers
- third-party integrations
- batch processing
- MCP connector
- consumer plans

The more agentic the architecture — the more of its data passes through these links.



A court order outlives your policy

NYT v. OpenAI (2025): the court ordered ALL ChatGPT logs preserved as evidence. The contractual "30 days" policy was powerless against a litigation hold from someone else's dispute.

Data that has left your perimeter lives by rules you do not control.

What to do: map the data per feature before production — what data, through which link, with what retention policy, which links are third-party. Regulated/sensitive data — only with ZDR/BAA or locally (on-prem).



Real coding agents — through the equipment frame.

The difference between tools is not “model quality”, but which equipment slots are filled and where the agent physically lives^[1].

< > Claude Code

terminal + IDE · proprietary

broad equipment: memory, instruction files, skills, full subagents, MCP — almost all 5 slots

the price — a lot of operational complexity

< > Aider

terminal-first · open source

minimal simplicity: no elaborate memory, no subagents, no skills. ~47k★ on GitHub

a “thin” rig is a deliberate choice, not an underdevelopment

< > Cursor

desktop IDE (a VS Code fork) · proprietary

the agent lives inside the editor — not a terminal. The form of integration is a separate axis from the rig

where the agent lives is also an architectural decision

< > OpenHands

platform on your own server · MIT · ~80k★

broad autonomy, local/Docker/cloud deployment

a working hypothesis by profile match — a likely candidate for “OpenClaw”, not a confirmed fact

Choosing a tool follows the same rule: don't take the most-equipped one by default — look at which rig your particular task needs.

^[1] *agent-harness-registry — a survey through the equipment frame*

Agent failures.

Each failure is a failure mode of the agent loop in a dated case: lesson + alternative.



1. Loop without limits

An agent told to "sync the orders" got HTTP 429
→ plan → call → 429 → ...

\$4,200 in 63 hours^[1]

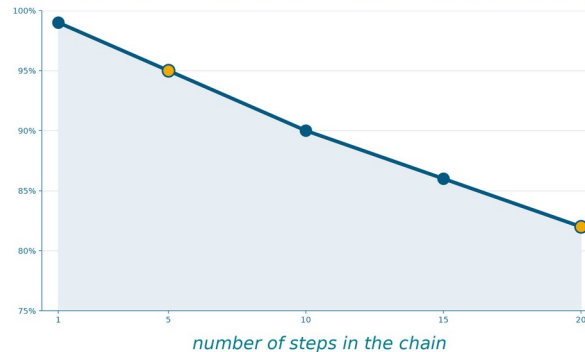
Baseline: a retry-with-backoff script would solve the same task in seconds-to-minutes, practically free

\$4,200 — the price not of automation, but of the wrong architecture choice for a predictable task

A more fitting architecture: a retry-with-backoff script, not an agent; budget and iteration limits OUTSIDE the agent

2. Compounding errors

5 × 99% ≈ 95%, 10 ≈ 90%, 20 ≈ 82% — reliabilities multiply



Takeaway: "tune one step" is a weak lever; "fewer transitions + a check between steps^[2]" is a strong one



3. Multi-agent fragility

dependent subtasks → parallel subagents make conflicting^[3] implicit decisions

A more fitting one: a single-threaded linear agent; multi-agent — only for broadly-parallel independent work

Agent failures are CLASSES, not a list of oddities.

Of **344** business-significant AI incidents, in **188** (≈55%) the autonomous system caused damage in production *WITHOUT* an attacker — the agent broke everything itself. Remember the class, not the date.

Failure class	Case · date	Lesson learned
Destruction + excess rights	PocketOS · 04.2026	System prompt ≠ control; hard boundary + least-privilege
Injection without a click (zero-click)	EchoLeak / M365 Copilot	The "lethal trifecta" = exploitable
Uncontrolled spend	\$48k/14h · \$1.3M/30d	No success criterion and no hard ceiling → the loop won't end
Package hallucination	slopsquatting · 19.7%	An invented package name → the attacker registers it; don't trust without a registry
Malicious MCP server	postmark-mcp · 09.2025	An MCP server = an unverified supply chain; version swap (rug-pull)
Multi-agent cascade	61% of cascades from upstream	Where it broke ≠ where it showed; errors multiply
Legal liability	OLG Hamm · Air Canada	"It was the bot's answer" — no defense; the output belongs to the company
Loop without a budget	\$4,200 / 63 h	"Try until it works" with no limit = literally
Error accumulation	reliability as p^n	p^n , not an average; "tune the model" is a weak lever
Multi-agent fragility	Cognition, 2025	For tasks with dependencies, multi-agent is worse than one

What to do: faced with a new incident — name ITS CLASS. The class sets the countermeasure (limit / least-privilege / check between steps), and the specific dates and amounts are not needed.

Four classes up close — with a comparison base.



PocketOS — a production DB in 9 seconds

The agent found a token with unlimited rights in someone else's file and deleted the volume + all backups. The nearest recoverable one was 3 months old.

Lesson: the system prompt is not a security control; you need a hard boundary.



Uncontrolled spend: \$48k / 14h · \$1.3M / 30d

A request with no success criterion — the planner expanded and never finished. Base: a normal session is cents-to-dollars; this is 3–5 orders of magnitude higher.

Lesson: a completion criterion + a hard ceiling + an emergency stop are mandatory.



Slopsquatting — 1 in 5 imports

Over 576,000 code samples, 19.7% of package references are hallucinations. The attacker registers the invented name → the next agent installs someone else's code.

Lesson: pin versions/hashes, lock files, review of new dependencies.



Cascade — 61% of errors from upstream

Across 73 incidents: in 61% the root was in an upstream layer (retrieval/plan), not where the failure became visible. The next agent takes the error as fact.

Lesson: checks between steps, fewer hops, traceability.

What to do: in all four the root is the same — the agent was applied without an external boundary (of rights, budget, validation). The boundary is placed OUTSIDE the agent; a prompt cannot replace it.

SECTION 5

How to choose: the decision framework

We went through every architecture separately — and where each one fails. Now we pull it together into one choice tool.

the choice tool · 4 cases

Section 0
Opening

Section 1
Prompt

Section 2
RAG

Section 3
Fine-tuning

Section 4
Agents

Section 5
Framework

5



How an engineer chose technology before the AI hype.

The architecture choice itself is not a new AI procedure, but a direct application of classical engineering-decision principles. The section's ladder and checklist stand on them.



From the requirement, not the tool

Requirements engineering: first pin down what the system must do, then pick the tool. Build-vs-buy: build your own or take a ready one.



KISS + YAGNI

Choose the simplest solution that meets the requirement; don't build in capacity "for the future" until a concrete requirement demands it.



Least power

From the W3C notes (Berners-Lee and Mendelsohn): take the least powerful of the sufficient tools — it's easier to analyze, verify and maintain.

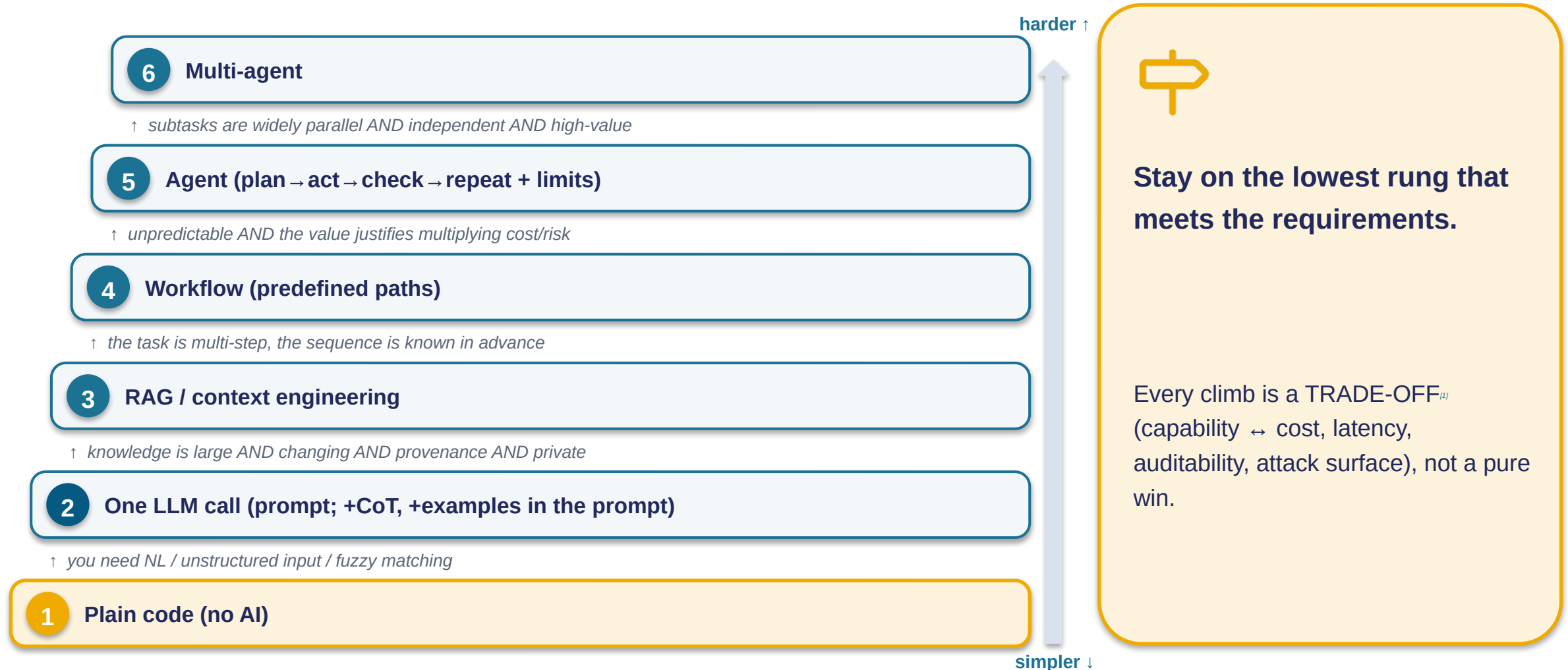
WHAT TO KEEP

The simplest sufficient architecture by default, and the burden of proof is on whoever wants to complicate it. In the AI era the principle isn't abolished, it gets costlier: an extra step adds non-determinism, token cost, latency, attack surface.

Bridge: the ladder of architectures (code → single call → RAG → workflow → agent → multi-agent) is the same least-power principle over AI architectures: stay on the lowest sufficient step, climb only to meet a requirement.

The ladder of architectural complexity.

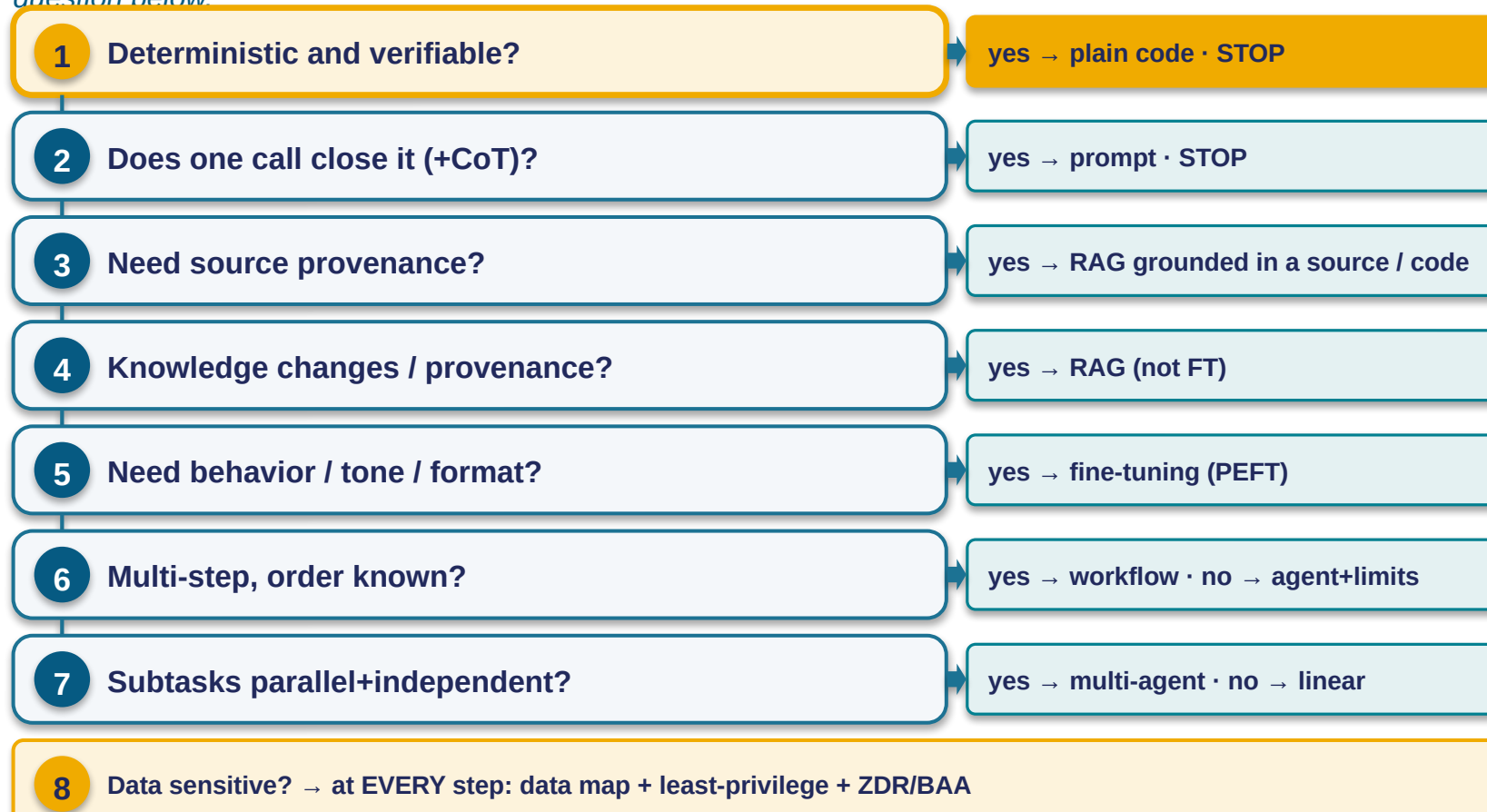
Stay on the bottom rung; climb only when the task requires it.



^[1] Anthropic — Building Effective Agents (the ladder rule) · The bottom rung is “plain code, no AI”: the ladder begins with the question “do we even need AI at all”.

The decision plan: a route of questions, not a sum of points.

Walk the task top to bottom; stop at the first question that fires^[1]. “Yes” → outcome on the right; “no” → next question below.



Example (task A)

“2000 regulations, changing weekly, an answer citing the clause” → question 4 (changing+provenance) is decisive → RAG with strict grounding in a source. Not fine-tuning (goes stale), not code (NL is needed).

Warm-up (task B)

“a bot over ~150 FAQ, changing once a quarter” — walk the route yourself; we go over it in the seminar.

Route priority: if the task is deterministic and verifiable — plain code, STOP here^[2]. AI would only add nondeterminism, cost, latency, and a surface for prompt injection.

The agent starter kit — and when to complicate it.

The same “don't complicate without a requirement” ladder, applied one level down — to a single agent's rig, not the whole system.



By default — a thin agent

- one instruction file
- flat memory
- NO subagents
- a minimal set of skills
- minimal MCP access

The burden of proof is on complication, not on simplicity.



A memory backend — when:

history has outgrown the context OR you need to search over facts. The same criterion as prompt → RAG.



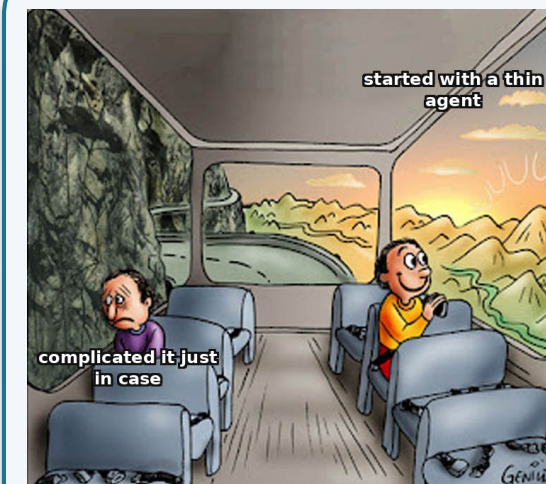
Subagents — when:

a subtask needs a separate window OR isolation of untrusted work (least-privilege).



More MCP access — when:

a specific task needs a specific tool — not “just in case”. Every connection is a trust boundary.



The same principle as the architecture ladder — applied to a single agent's rig: the presence paradox showed that even an instruction file “as a ritual” doesn't work. Complicate against a specific verifiable trigger.

Takeaway: mechanism → its boundary → what to do.

Mechanism	Where it breaks (boundary)	What to do
Prompt / role	a role-persona changes tone, not accuracy	accuracy — via context and RAG, not via "you are an expert"
Chain-of-thought	faithfulness is low (~25–39%)	check the result, not the self-explanation
RAG	"retrieved" ≠ "retrieved correctly"	grounding in a source + retrieval metrics + "I don't know" as the norm
Fine-tuning / PEFT	changes behavior, not knowledge; forgetting	PEFT + a check loop + versioning; knowledge → RAG
Agent loop	plan → act → check → repeat — 4 points of failure	check against an external criterion; a hard ceiling on the loop
Equipment / memory	each slot is a trade-off, not a win	add a slot on demand, with a check
Multi-agent	p ⁿ : 95%×10 ≈ 60%; +15× tokens	one strong agent by default
Security	injection × broad rights; ZDR isn't everything	least-privilege + a per-feature data map
"Not AI at all"	deterministic + verifiable	plain code — cheaper, more predictable, auditable

To know a tool is to know its boundaries. Choosing an architecture = find the lowest rung that meets the task's requirement.

Human validator + the MIT NANDA lesson.



The agent acts — a human checks the result and the facts

against an independent source of truth, not by the plausibility of the reasoning^[1]; the model's self-explanation ≠ control.

Degree of autonomy

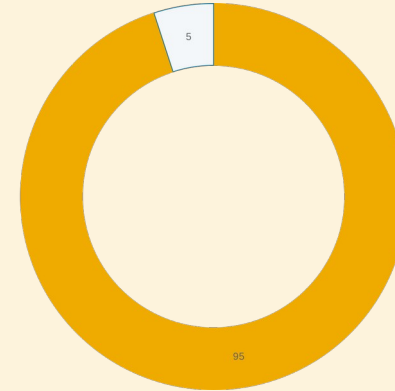
from “a human presses the button” to “the agent notifies after the fact”

Trust scope

reads (easy to roll back) vs writes · reversible vs irreversible

Continuous monitoring

quality metrics all the time, not a one-off check at the start



~95%

of enterprise GenAI pilots with no measurable return on investment — the root is in the learning gap and the integration failure, not model quality^[2]

“Launch AI” ≠ “get value”. Architectural-integration discipline decides. Sometimes the right answer is the simplest architecture or no AI.

Next — industries. Start: software development, the SDLC × AI axis.

Lecture 4 takes the same apparatus and lays it out across the software lifecycle stages. Each stage leans on an anchor from this lecture:

Requirements / design

picking an architecture per step → the 6-rung ladder

Coding

coding agents through the equipment frame; failures as a CLASS

Testing

the trust boundary and least-privilege; data outside ZDR

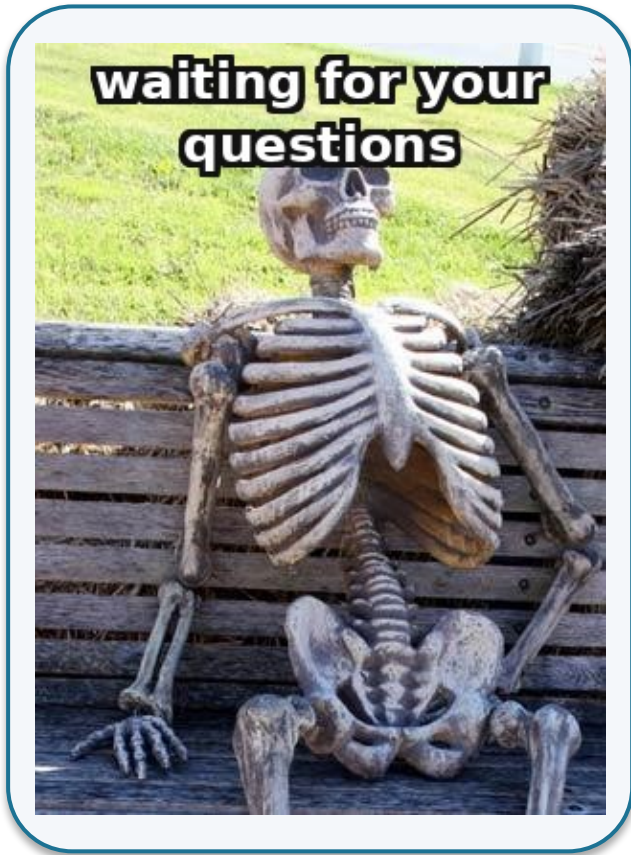
Operations

a human validator checks the result, not the self-explanation

Assignment — Seminar 3: run the architecture-choice checklist on 3 cases (chat / agent / RAG / API).

What to do: this frame is the base for Lectures 4–17. On each industry lecture, run the same choice checklist.





Q&A

Thank you